

Presented by Murat
Khamatyanov 24B44

UNSAFE CODE, POINTER TYPES, AND FUNCTION POINTERS

```
<PropertyGroup>  
  <OutputType>Exe</OutputType>  
  <TargetFramework>net10.0</TargetFramework>  
  <AllowUnsafeBlocks>true</AllowUnsafeBlocks>  
  <ImplicitUsings>enable</ImplicitUsings>  
  <Nullable>enable</Nullable>  
</PropertyGroup>
```

C#

 Copy

```
type* identifier;  
void* identifier; //allowed but not recommended
```

C#

 Copy

```
int* p1, p2, p3;    // Ok  
int *p1, *p2, *p3;  // Invalid in C#
```

- `int* p`: `p` is a pointer to an integer.
- `int** p`: `p` is a pointer to a pointer to an integer.
- `int*[] p`: `p` is a single-dimensional array of pointers to integers.
- `char* p`: `p` is a pointer to a char.
- `void* p`: `p` is a pointer to an unknown type.

```
unsafe
{
    byte[] bytes = [1, 2, 3];
    fixed (byte* pointerToFirst = bytes)
    {
        Console.WriteLine($"The address of the first array element: {(long)pointerToFirst:X}.");
        Console.WriteLine($"The value of the first array element: {*pointerToFirst}.");
    }
}

// Output is similar to:
// The address of the first array element: 2173F80B5C8.
// The value of the first array element: 1.
```

C#

 Copy

```
unsafe
{
    int[] numbers = [10, 20, 30];
    fixed (int* toFirst = &numbers[0], toLast = &numbers[^1])
    {
        Console.WriteLine(toLast - toFirst); // output: 2
    }
}
```

C#

 Copy

```
unsafe
{
    int[] numbers = [10, 20, 30, 40, 50];
    Span<int> interior = numbers.AsSpan()[1..^1];
    fixed (int* p = interior)
    {
        for (int i = 0; i < interior.Length; i++)
        {
            Console.Write(p[i]);
        }
        // output: 203040
    }
}
```

C#

 Copy

```
unsafe
{
    var message = "Hello!";
    fixed (char* p = message)
    {
        Console.WriteLine(*p); // output: H
    }
}
```

Operator/Statement	Use
<code>*</code>	Performs pointer indirection.
<code>-></code>	Accesses a member of a struct through a pointer.
<code>[]</code>	Indexes a pointer.
<code>&</code>	Obtains the address of a variable.
<code>++</code> and <code>--</code>	Increments and decrements pointers.
<code>+</code> and <code>-</code>	Performs pointer arithmetic.
<code>==</code> , <code>!=</code> , <code><</code> , <code>></code> , <code><=</code> , and <code>>=</code>	Compares pointers.
<code>stackalloc</code>	Allocates memory on the stack.
<code>fixed statement</code>	Temporarily fixes a variable so that its address can be found.

```
int number = 1024;

unsafe
{
    // Convert to byte:
    byte* p = (byte*)&number;

    System.Console.Write("The 4 bytes of the integer:");

    // Display the 4 bytes of the int variable:
    for (int i = 0 ; i < sizeof(int) ; ++i)
    {
        System.Console.Write(" {0:X2}", *p);
        // Increment the pointer:
        p++;
    }
    System.Console.WriteLine();
    System.Console.WriteLine($"The value of the integer: {number}");

    /* Output:
        The 4 bytes of the integer: 00 04 00 00
        The value of the integer: 1024
    */
}
```



```
internal unsafe class Example
{
    1 reference
    internal unsafe struct Buffer
    {
        public fixed char fixedBuffer[128];
    }

    public Buffer buffer = default;

    0 references
    public void AccessEmbeddedArray()
    {
        unsafe
        {
            // Pin the buffer so the GC can't move it
            fixed (char* charPtr = buffer.fixedBuffer)
            {
                *charPtr = 'A';
            }

            // Safe access via indexer
            char c = buffer.fixedBuffer[0];
            Console.WriteLine(c);

            // Modify via indexer
            buffer.fixedBuffer[0] = 'B';
            Console.WriteLine(buffer.fixedBuffer[0]);
        }
    }
}
```

```
static unsafe void Copy(byte[] source, int sourceOffset, byte[] target,
    int targetOffset, int count)
{
    fixed (byte* pSource = source, pTarget = target)
    {
        // Copy the specified number of bytes from source to target.
        for (int i = 0; i < count; i++)
        {
            pTarget[targetOffset + i] = pSource[sourceOffset + i];
        }
    }
}
```

```
unsafe class temp  
{
```

0 references

```
public static T Combine<T>(Func<T, T, T> combinator, T left, T right) =>  
    combinator(left, right);
```

1 reference

```
public static unsafe T UnsafeCombine<T>(delegate*<T, T, T> combinator, T left, T right)  
    where T : unmanaged =>  
    combinator(left, right);
```

0 references

```
public unsafe void run()  
{  
    int product = 0;  
  
    static int localMultiply(int x, int y) => x * y;  
  
    product = UnsafeCombine(&localMultiply, 3, 4);  
    Console.WriteLine(product);  
}  
}
```

1 reference

`static class NativeMethods``{``[DllImport("user32.dll", CharSet = CharSet.Unicode)]`

1 reference

`public static extern int MessageBox(``IntPtr hWnd,``string text,``string caption,``uint type);``}`

0 references

`class Program``{`

0 references

`static void Main()``{``NativeMethods.MessageBox(IntPtr.Zero, "Hi", "Title", 0);``}``}`